R4DS 14 – Iteration & Base R

Modifying Multiple Columns

`across()` — Apply to Many Columns

Instead of repeating the same operation per column:

```
df |> summarize(a = median(a), b = median(b),
                c = median(c), d = median(d))
```

Use `across(.cols, .fns)`:

```
df |> summarize(across(a:d, median))
```

Common `.cols` selectors:

Selector	Matches
`a:d`	Columns a through d
`everything()`	All non-grouping columns
`where(is.numeric)`	All numeric columns
`starts_with("x")`	Columns starting with "x"

Anonymous Functions in `across()`

Pass the **function name**, not a call. For extra arguments, use an anonymous function:

```
df_miss <- tibble(a = c(1, NA, 3, 4),
  b = c(NA, 5, 6, 7), c = c(7, 8, NA, 10))

# Without na.rm: NAs propagate
df_miss |> summarize(across(a:c, median))
```

```
#> # A tibble: 1 x 3
#>       a     b     c
#>   <dbl> <dbl> <dbl>
#> 1    NA    NA    NA
```

```
# Fix: anonymous function
df_miss |> summarize(
  across(a:c, \(x) median(x, na.rm = TRUE)))
```

```
#> # A tibble: 1 x 3
#>       a     b     c
#>   <dbl> <dbl> <dbl>
#> 1     3     6     8
```

`\(x)` is shorthand for `function(x)`.

Multiple Functions and `.names`

Pass a **named list** of functions:

```
df_miss |> summarize(across(a:c, list(
  med = \(x) median(x, na.rm = TRUE),
  n_miss = \(x) sum(is.na(x)))))
```

```
#> # A tibble: 1 x 6
#>   a_med a_n_miss b_med b_n_miss c_med c_n_miss
#>   <dbl>    <int> <dbl>    <int> <dbl>    <int>
#> 1     3        1     6        1     8        1
```

Control output names with `.names` (glue syntax):

```
df_miss |> summarize(across(a:c, list(
  med = \(x) median(x, na.rm = TRUE),
  n_miss = \(x) sum(is.na(x))),
  .names = "{.fn}_{.col}"))
```

Placeholders: `{.col}` = column name, `{.fn}` = function name.

if_any() and if_all()

Filter rows based on conditions across multiple columns:

```
# Rows where ANY column has NA (OR logic)
df_miss |> filter(if_any(a:c, is.na))
```

```
#> # A tibble: 3 x 3
#>       a     b     c
#>   <dbl> <dbl> <dbl>
#> 1     1    NA     7
#> 2    NA     5     8
#> ... [output truncated]
```

```
# Rows where ALL columns are non-NA (AND logic)
df_miss |> filter(if_all(a:c, \(x) !is.na(x)))
```

```
#> # A tibble: 1 x 3
#>       a     b     c
#>   <dbl> <dbl> <dbl>
#> 1     4     7    10
```

Use in `mutate()` with `coalesce()` to replace NAs:

```
df_miss |> mutate(across(a:c, \(x) coalesce(x, 0)))
```

across() in Functions

Combine embracing with tidy-select:

```
summarize_means <- function(df,
    summary_vars = where(is.numeric)) {
  df |> summarize(
    across({{ summary_vars }},
      \(x) mean(x, na.rm = TRUE)),
    n = n(), .groups = "drop")
}

diamonds |> group_by(cut) |> summarize_means()
diamonds |> group_by(cut) |>
  summarize_means(c(carat, x:z))
```

Default `where(is.numeric)` selects all numeric columns; user can override.

Exercises (across)

1. Practice `across()` skills:
   a. Compute unique values per column in `palmerpenguins::penguins`
   b. Compute mean of every column in `mtcars`
   c. Group `diamonds` by `cut`, `clarity`, `color`; count and compute mean of numeric columns
2. What happens with an unnamed list of functions in `across()`?
3. Modify `expand_dates()` to auto-remove the original date columns after expansion.
4. Explain each step:

```
show_missing <- function(df, group_vars,
    summary_vars = everything()) {
  df |> group_by(pick({{ group_vars }})) |>
    summarize(across({{ summary_vars }},
      \(x) sum(is.na(x))), .groups = "drop") |>
    select(where(\(x) any(x > 0)))
}
```

Solutions (across) — 1

```
# 1b. Mean of every column in mtcars
mtcars |>
  summarize(across(everything(), mean))
```

```
#>        mpg    cyl     disp       hp     drat      wt
#> 1 20.09062 6.1875 230.7219 146.6875 3.596563 3.21725
#>       qsec     vs      am   gear   carb
#> 1 17.84875 0.4375 0.40625 3.6875 2.8125
```

```
# 1c. Grouped means of numeric columns
diamonds |> group_by(cut, clarity, color) |>
  summarize(n = n(),
    across(where(is.numeric), mean),
    .groups = "drop")
```

```
#> # A tibble: 276 x 11
#>    cut   clarity color     n carat depth table price
#>    <ord> <ord>   <ord> <dbl> <dbl> <dbl> <dbl> <dbl>
#>  1 Fair  I1      D         4 1.88   65.6  56.8 7383
#>  2 Fair  I1      E         9 0.969  65.6  58.1 2095.
#> ... [output truncated]
```

1a. Same pattern: `penguins |> summarize(across(everything(), n_distinct))`

2. Unnamed functions get numeric suffixes: `col_1`, `col_2`, etc.
3. Add `select(!where(is.Date))` at the end:

```
expand_dates <- function(df) {
  df |>
    mutate(across(where(is.Date),
      list(year = year, month = month,
           day = mday))) |>
    select(!where(is.Date))
}
```

4. Groups by `group_vars`, counts NAs per `summary_vars` column, then `select(where(...))` keeps only columns where **any group has missing values**.

Reading Multiple Files

map() and list_rbind()

```
paths <- list.files("data/gapminder",
  pattern = "[.]xlsx$", full.names = TRUE)
```

`map()` applies a function to each element, returning a **list**:

```
files <- map(paths, readxl::read_excel)
files[[1]]      # access first data frame
```

`list_rbind()` stacks all data frames into one:

```
paths |>
  map(readxl::read_excel) |>
  list_rbind()
```

Pass extra arguments with an anonymous function:

```
paths |>
  map(\(p) readxl::read_excel(p, n_max = 1)) |>
  list_rbind()
```

Data in the Path

File names often contain data. Use `set_names()` + `names_to`:

```
paths |>
  set_names(basename) |>
  map(readxl::read_excel) |>
  list_rbind(names_to = "year") |>
  mutate(year = parse_number(year))
```

Workflow: list files → name them → read each → bind with name column → parse.

Save processed data to avoid re-reading:

```
gapminder <- paths |>
  set_names(basename) |>
  map(readxl::read_excel) |>
  list_rbind(names_to = "year") |>
  mutate(year = parse_number(year))
write_csv(gapminder, "gapminder.csv")
```

Handling Failures with `possibly()`

`possibly()` wraps a function to return a default instead of erroring:

```
files <- paths |>
  map(possibly(
    \(p) readxl::read_excel(p), NULL))

data <- files |> list_rbind()

# Find which files failed
failed <- map_vec(files, is.null)
paths[failed]
```

Useful when some files are corrupt or have unexpected formats.

Saving Multiple Outputs

Writing Files with `walk()`

`walk()` is like `map()` but for **side effects** (writing, printing):

```
by_clarity <- diamonds |> group_nest(clarity)
by_clarity
```

```
#> # A tibble: 8 x 2
#>   clarity               data
#>   <ord>   <list<tibble[,9]>>
#> 1 I1                [741 x 9]
#> 2 SI2             [9,194 x 9]
#> ... [output truncated]
```

```
by_clarity <- by_clarity |>
  mutate(path = str_glue("diamonds-{clarity}.csv"))

# Write each subset to its own file
walk2(by_clarity$data, by_clarity$path, write_csv)
```

`walk2()` maps over **two** inputs simultaneously.

Saving Plots

```
carat_histogram <- function(df) {
  ggplot(df, aes(x = carat)) +
    geom_histogram(binwidth = 0.1)
}

by_clarity <- by_clarity |>
  mutate(
    plot = map(data, carat_histogram),
    path = str_glue("clarity-{clarity}.png"))

walk2(by_clarity$path, by_clarity$plot,
  \(path, plot) ggsave(path, plot,
    width = 6, height = 6))
```

Pattern: nest data → create plots with `map()` → save with `walk2()`.

Base R Essentials

Subsetting Vectors with [

Five ways to subset a vector:

```
x <- c("one", "two", "three", "four", "five")
x[c(3, 2, 5)]           # positive integers
```

```
#> [1] "three" "two"   "five"
```

```
x[c(-1, -3, -5)]        # negative = exclude
```

```
#> [1] "two"  "four"
```

```
x <- c(10, 3, NA, 5, 8, 1, NA)
x[!is.na(x)]            # logical vector
```

```
#> [1] 10  3  5  8  1
```

```
x[x %% 2 == 0]          # NAs propagate!
```

```
#> [1] 10 NA  8 NA
```

```
x <- c(abc = 1, def = 2, xyz = 5)
x[c("xyz", "def")]      # by name
```

```
#> xyz def
#>   5   2
```

Also: repeated indices (`x[c(1, 1, 5)]`) and empty (`x[]`) are valid

Subsetting Data Frames with [

`df[rows, cols]` — rows and columns independently:

```
df <- tibble(x = 1:3, y = c("a", "e", "f"),
             z = runif(3))
df[1, 2]                  # row 1, col 2
```

```
#> # A tibble: 1 x 1
#>   y
#>   <chr>
#> 1 a
```

```
df[df$x > 1, ]            # filter rows
```

```
#> # A tibble: 2 x 3
#>       x y         z
#>   <int> <chr> <dbl>
#> 1     2 e     0.111
#> 2     3 f     0.346
#> ... [output truncated]
```

`df[, c("x", "y")]` selects columns. **Tibble vs. data.frame:** tibbles always return tibbles; `data.frame` may drop to a vector.

dplyr Equivalents

dplyr	Base R
`filter(df, x > 1)`	`df[df$x > 1, ]`
`arrange(df, x)`	`df[order(df$x), ]`
`select(df, x, z)`	`df[, c("x", "z")]`
`mutate(df, z = x+y)`	`df$z <- df$x + df$y`
`pull(df, x)`	`df$x` or `df[["x"]]`

`subset()` combines filter + select:

```
df |> subset(x > 1, c(y, z))
# same as: df |> filter(x > 1) |> select(y, z)
```

Single Elements: [[and $

[[extracts a **single element**; $ is shorthand by name:

```
tb <- tibble(x = 1:4, y = c(10, 4, 1, 21))
tb[["x"]]       # by name
```

```
#> [1] 1 2 3 4
```

```
tb$x            # shorthand
```

```
#> [1] 1 2 3 4
```

```
tb[[1]]         # by position
```

```
#> [1] 1 2 3 4
```

Create new columns with $:

```
tb$z <- tb$x + tb$y
tb
```

```
#> # A tibble: 4 x 3
#>       x     y     z
#>   <int> <dbl> <dbl>
#> 1     1    10    11
#> 2     2     4     6
#> ... [output truncated]
```

Lists: [vs. [[

```
l <- list(a = 1:3, b = "a string",
          c = pi, d = list(-1, -5))
```

`[` returns a **sub-list**; `[[` extracts the **element itself**:

```
str(l[1:2])     # sub-list of first 2
```

```
#> List of 2
#>  $ a: int [1:3] 1 2 3
#>  $ b: chr "a string"
```

```
str(l[[1]])     # the vector itself
```

```
#>  int [1:3] 1 2 3
```

```
str(l[[4]])     # extracts the inner list
```

```
#> List of 2
#>  $ : num -1
#>  $ : num -5
```

Same for data frames: `df["x"]` = one-column data frame; `df[["x"]]` = vector.

Exercises (Base R)

1. Create functions that take a vector and return:
   a. Elements at even-numbered positions
   b. Every element except the last
   c. Only even values (no missing values)
2. Why is `x[-which(x > 0)]` not the same as `x[x <= 0]`?
3. What happens with `[[` and an index larger than the vector length? With a non-existent name?

Solutions (Base R)

1.

```
even_pos <- function(x) x[seq(2, length(x), 2)]
drop_last <- function(x) x[-length(x)]
even_vals <- function(x) x[!is.na(x) & x %% 2 == 0]

even_pos(1:10)
```

```
#> [1]  2  4  6  8 10
```

```
drop_last(1:5)
```

```
#> [1] 1 2 3 4
```

```
even_vals(c(1, NA, 4, 7, 8))
```

```
#> [1] 4 8
```

2. With `NA`: `x <= 0` returns `NA` (keeps element as `NA`), but `which()` drops `NA`s. Also, `-which(x > 0)` on empty result gives `-integer(0)` = select nothing.
3. `[[` with out-of-bounds index: error. Non-existent name: `NULL` for lists, error for atomic vectors.

Apply Family and For Loops

The Apply Family

Function	Like purrr	Returns
`lapply(x, f)`	`map(x, f)`	Always a list
`sapply(x, f)`	—	Simplified (unpredictable)
`vapply(x, f, type)`	`map_vec(x, f)`	Specified type (strict)
`tapply(x, grp, f)`	grouped summarize	Named vector

```
sapply(mtcars[1:4], is.numeric)
```

```
#>  mpg  cyl disp   hp
#> TRUE TRUE TRUE TRUE
```

```
tapply(diamonds$price, diamonds$cut, mean)
```

```
#>      Fair      Good Very Good   Premium     Ideal
#>  4358.758  3928.864  3981.760  4584.258  3457.542
```

Prefer `vapply()` over `sapply()` — it fails clearly on type mismatches.

For Loops

```
for (element in vector) {
  # do something with element
}
```

Use for **side effects** (same purpose as `walk()`):

```
# purrr
paths |> walk(append_file)

# base R equivalent
for (path in paths) {
  append_file(path)
}
```

For loops themselves aren't slow — **growing vectors** inside them is.

Pre-allocation and `seq_along()`

Never grow a vector in a loop — pre-allocate:

```
# Good: pre-allocate, then fill
files <- vector("list", length(paths))
for (i in seq_along(paths)) {
  files[[i]] <- readxl::read_excel(paths[[i]])
}
do.call(rbind, files)  # combine list

# Bad: grows quadratically (O(n^2))
out <- NULL
for (path in paths) {
  out <- rbind(out, readxl::read_excel(path))
}
```

`seq_along()` generates indices; `do.call(rbind, ...)` combines a list.